

OSAT SemiGuard

Architecture & Version History

0.2.6

A plain-English guide for student engineers

Current architecture + cumulative changelog from 0.2.6

FROZEN 0.2.6 APPLICATION CODE

Understand the full system, find your team's boundary, and follow measurements into evidence.

The demonstration is synthetic. Real OSAT validation, prospective plant validation and production qualification remain false.

PRE-STEPS

Identity & trust

STEPS

Analysis & maintenance

POST-STEPS

Proof & research

CONTENTS

Find your route

Read from the top for a full introduction, or use the clickable contents and PDF bookmarks to jump to one responsibility.

01 What SemiGuard is for	3
02 The architecture at a glance	4
03 PRE-STEPS: identity, meaning and trust	5
04 The nine station families	8
05 STEPS01-03: physics and useful features	9
06 STEPS04-05: optional family-level research	12
07 STEPS06-07: the exact-machine baseline	14
08 STEP08: telemetry enters and is stored	16
09 STEPS09-10: health and fault evidence	17
10 STEPS11-15: maintenance responsibilities	19
11 POST01-02: demonstrations and frozen replay	25
12 POST03-04: external scientific evidence	27
13 POST05: the controlled end-to-end proof	29
14 Execution order: prepare the model	31
15 Execution order: monitor repeatedly	32
16 The five screens and headless commands	33
17 How the five student teams work together	34
18 Find your team's first files	35
19 Repository boundaries, checks and release identity	36
20 A short glossary	37
21 Changelog from 0.2.6	38

A useful first pass

- New to the system: chapters 01-03, then execution in 14-15.
- Working on a contribution: your team in 17-18, then its stage and tests.
- Reviewing a version: chapters 13, 19 and the changelog in 21.

CHAPTER 01 / CURRENT ARCHITECTURE

What SemiGuard is for

OSAT SemiGuard explores a practical question: when trustworthy machine measurements change, can the system show useful equipment-health evidence and explain why it deserves attention?

OSAT means outsourced semiconductor assembly and test. This project concerns equipment used around assembly, packaging and testing. It does not decide whether a semiconductor product passes its quality requirements.

The current version

Version 0.2.6 is a controlled end-to-end proof of concept. Its demonstrations use authored synthetic data. The system connects data checks, physical relationships, machine-specific comparison, health states, fault evidence, maintenance records and a user interface.

CODE FROZEN: this guide describes the reviewed 0.2.6 implementation. Adding the guide does not change application behavior.

What a successful demonstration means

A successful run shows that the parts work together under the declared demonstration conditions. It can show a new machine starting as UNKNOWN, accepting a separately validated nominal baseline, detecting a controlled change, creating one demo ticket and recovering without closing that ticket automatically.

That success does not establish real OSAT performance, prospective plant validation, production qualification, a physical root cause or a calibrated failure probability. All three real-OSAT, prospective-plant and production-validation flags remain false.

How to use this guide

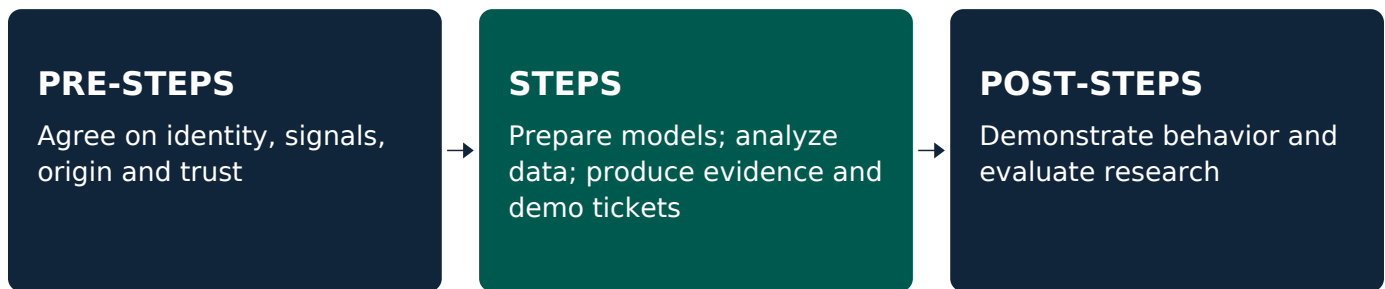
Read chapter 02 for the overall map. Chapters 03-13 explain PRE-STEPS, STEPS and POST-STEPS in numerical order. Chapters 14-15 walk through preparation and monitoring in execution order; chapter 16 explains the screens. Chapters 17-18 locate your team, chapter 19 covers contribution checks and chapter 21 records version history.

Each stage explains its purpose, inputs, work, outputs, examples, limits and contributor entry. Read one stage's page without learning every implementation first. Additional pages keep the approved text size and layout. File paths are relative to the extracted project folder; stage paths begin at `osat_edge/roadmap/`. The Python package remains `osat_edge`.

CHAPTER 02 / CURRENT ARCHITECTURE

The architecture at a glance

Read the stage catalog in order: PRE01-PRE03, STEP01-STEP15, then POST01-POST05. These three layers group responsibilities. The preparation and monitoring walkthroughs afterward show when each responsibility is used.



PRE-STEPS establish the meaning of the inputs

Before a number becomes evidence, the system needs to know which installed machine produced it, what signal it represents, its unit, its time and its declared data origin. PRE01 through PRE03 own these contracts and checks.

STEPS do preparation and monitoring

Prepare first: define physical relationships, collect nominal data, fit and separately validate the machine model. Then monitor: read measurements, calculate features, compare the machine, determine health and build maintenance evidence. Monitoring uses the fitted model; it does not refit it every tick.

POST-STEPS observe and test

POST01 and POST02 exercise the operational system using synthetic demonstrations and a frozen replay. POST03 and POST04 perform isolated external-data research. POST05 orchestrates the controlled full proof. It observes decisions made by existing stages rather than implementing another health algorithm.

The user interface sits beside these layers

The UI displays pipeline results and research information. It does not choose health thresholds, fit models or grant maintenance authority. The headless CLI can run the demonstrations and research commands without launching the UI.

The main connector

`osat_edge/pipeline.py` connects operational stages through `MachinePipeline`. `FleetPipeline` coordinates the demonstration machines. Ordinary Python functions and small data records carry results between owners. Stage IDs name modules, not positions on a clock: STEP08 must supply data before STEPS02-03 analyze it. Chapters 14-15 show the actual dependency order.

CHAPTER 03 / CURRENT ARCHITECTURE

PRE-STEPS: identity, meaning and trust

PRE01 - shared contracts and authority limits

PRE01 supplies the shared vocabulary for machines, measurements and results. A contract is a small record with named fields and rules, letting teams exchange data without knowing each other's internal code.

What enters

Contributors declare an exact installed machine, its equipment family and station, approved measurement channels, timestamped samples and operating context. A channel includes its name, unit, subsystem, required/optional status, expected sampling period, stale limit and source ID. A sample contains the machine ID, channel, time, value, unit and source ID.

What happens

`contracts.py` defines these records and named values. Time must include a timezone and is normalized to UTC, so two computers do not disagree about when an event happened. Samples and windows reject non-finite values. Window arrays must align and have strictly increasing timestamps; their stored copies are read-only.

The records distinguish operating state from health. PROCESSING describes activity; NORMAL describes an assessment. They also distinguish data origin from execution mode: SYNTHETIC names evidence, while REAL_REPLAY names how input is played back. These distinctions remain attached as evidence moves through the system.

What leaves

Other stages receive consistent identities, channel definitions, samples, windows and named states. PRE01 does not itself check every source mapping or decide health. PRE02 supplies the station definitions; PRE03 verifies provenance; STEP08 checks incoming telemetry against the declarations.

Example and limits: Two wafer saws can share the same family and channel names while having different machine IDs. Their telemetry and fitted baselines must remain separate. Likewise, an unfamiliar IDLE context may produce UNKNOWN rather than a health alarm. `authority.py` defines SHADOW / READ-ONLY restrictions: the project cannot issue equipment-control, shutdown, recipe-change or process-control actions.

Contributor entry: Data Pipeline: read `pre_steps/pre01_common/contracts.py`, `authority.py` and stage tests. Future first task: explain a rejected timestamp or identity.

A record provides an agreed meaning. It does not turn a declared origin into verified provenance or grant operational authority.

CHAPTER 03 / CURRENT ARCHITECTURE / CONTINUED

PRE-STEPS: identity, meaning and trust

PRE02 - the station and channel registry

PRE02 is the approved lookup list for equipment and signals. It tells later stages what a station and channel mean, preventing a similarly named source column from silently becoming the wrong engineering measurement.

What enters

The registry contains reviewed equipment and measurement declarations for the nine station families. A station profile has a family, station ID, display name and channel specifications. The exact installed-machine identity is supplied separately; a station profile describes a kind of equipment rather than one unit's learned behavior.

What happens

`registry.py` builds **STATIONS**, the shared station lookup. Each channel names its subsystem, unit, source ID, sampling period, stale limit and whether it is required. A station rejects duplicate channel names and duplicate source IDs. Its subsystem list is derived from its channels, keeping those two views consistent.

STEP08 uses the profile to check incoming measurements. STEP02 uses it to name features and attach them to subsystems. STEP09 uses it to identify subsystems that need scoreable evidence. The UI can display the same station meaning without maintaining another independent list.

What leaves

Consumers receive canonical station and channel definitions. Canonical means the agreed project name and interpretation. The registry does not read equipment, convert an arbitrary vendor export automatically, choose a model or prove that a sensor is installed. A source adapter must still provide an explicit approved mapping.

Example and limits: For WS-01, spindle current and spindle speed describe distinct quantities with distinct units and approved sources. A voltage column cannot be accepted as current because its name looks familiar. A future mapping must establish what the source actually measures. Required/optional also has a precise meaning: an unavailable optional signal can remain unavailable, while missing required evidence can block an assessment.

Contributor entry: Data Pipeline and Failure Research: read `pre_steps/pre02_machine_registry/registry.py` and boundary tests. Future first task: compare one declaration with its source mapping. Chapter 04 lists families and fleet scope.

An approved signal definition is a data contract. It is not a claim that every listed station has validated failure detection.

CHAPTER 03 / CURRENT ARCHITECTURE / CONTINUED

PRE-STEPS: identity, meaning and trust

PRE03 - provenance and source verification

PRE03 checks where evidence came from, which bytes were used and what signals and labels mean. A plant-like filename or replay mode cannot grant REAL_OSAT status.

What enters

A declaration names the citation, pinned bytes, reviewed OSAT provenance basis, machine/equipment, boundary meaning, label types and evidence class. Canonical execution also requires a station ID and exact unit/measurement mappings.

What happens

`provenance.py` validates the declaration and compares supplied local bytes with the pinned identity. Mappings must be unambiguous; a known canonical station and matching units are required when mappings are supplied. Label meanings remain distinct: a machine alarm, product-quality result and adjudicated equipment fault are different kinds of evidence.

Multi-file identity uses relative paths and explicit content boundaries, rejecting invalid/duplicate names. Legacy formats only reproduce frozen external evidence, not new REAL_OSAT declarations.

What leaves

Matching bytes and a valid declaration produce a verified source record. Mismatch rejects the source. Without executable canonical mappings, its use remains limited. Verification does not prove sensor accuracy or scientific truth.

Keep two questions separate

Question	Named values	Meaning
Where did the evidence come from?	SYNTHETIC / EXTERNAL_BENCHMARK / REAL_OSAT	Authored simulation, isolated external research, or explicitly verified OSAT evidence.
How is the program running it?	SIMULATION / REAL_REPLAY / LIVE_EQUIPMENT	Generated input, replayed input, or incoming equipment-like input.

Example: SYNTHETIC input remains synthetic when replayed. Local simulation is not plant evidence.

Contributor entry: Data Pipeline and Reliability: read `pre_steps/pre03_data_provenance/provenance.py` and tests. Future first task: explain a rejected mapping or an alarm/fault label distinction.

Verified input does not prove a valid model or authorize maintenance.

CHAPTER 04 / CURRENT ARCHITECTURE

The nine station families

A family groups equipment with a common role. An exact machine is one identified installed unit. A subsystem is a named part of that equipment, such as a spindle, cooling system or motor. A channel is one measurement associated with a subsystem.

The registry is the source of truth for the following station IDs and approved signal names. The examples below help you navigate; they do not assert validated fault detection for every family.

Station	Equipment role	Example registered measurements
WM-01	Wafer Mount	Vacuum pressure, roller current, web tension
WS-01	Wafer Saw / Dicing	Spindle current, speed, vibration; coolant pressure
DA-01	Die Attach	Nozzle vacuum, axis current, position error
WB-04	Wire Bond	Bond force, head current, ultrasonic measurements
MO-01	Molding	Cavity pressure, transfer current, mold temperature
MK-01	Laser Marking	Delivered laser power, drive current, tracking error
TF-01	Trim / Form	Punch force, press current, die vibration
SG-01	Singulation	Blade vibration, spindle current, coolant flow
FT-01	Final Test	Contact voltage drop, site current, socket temperature

Required and optional signals

A required channel must provide enough current usable evidence for the relevant analysis. A missing optional channel is not silently replaced with a made-up value; its evidence can remain unavailable. Units and expected sampling periods are part of the registry.

Equipment state is a separate concept

PROCESSING, IDLE, SETUP and other operating states describe what the equipment is doing. NORMAL, WATCH and other health states describe the available health assessment. An IDLE machine is not automatically unhealthy; a model without an IDLE baseline can instead report UNKNOWN.

Current fleet scope

The demonstration has one machine per family and stores these machines by family. This release does not represent several simultaneous installed machines of the same family in that fleet container. Historical family-model training can use multiple machines. Exact-machine model and telemetry checks still enforce installed-machine identity.

CHAPTER 05 / CURRENT ARCHITECTURE

STEPS01-03: physics and useful features

STEP01 - a reviewed physics library

STEP01 records physical hypotheses, equations, measurement requirements and evidence limits. It gives contributors an engineering reason to test a feature instead of treating every unusual number as a failure.

What enters

Failure Research investigates failure modes per machine, including submachines, physical mechanisms, sensors and parameters. Its dossier explains whether each problem could potentially be modeled, what measurements are needed, normal lookalikes and a test plan. A relation also needs units, an applicable range and parameters requiring calibration. Reference identity and uncertainty remain essential evidence.

What happens

`library.py` exposes the reviewed library. `families/` owns equipment-specific relationships; `core/` owns records and reusable checks for references, calibration, units, uncertainty, diagnostics and evidence maturity. Research dossiers retain the reasoning behind each proposed relationship.

The library separates runtime relationships, research-only candidates and rejected claims. A useful paper does not automatically make its equation usable with this project's sensors. A relation must name what is actually measured and what it can support. Missing sensors, unsupported assumptions and normal operating effects remain visible.

What leaves

STEP03 can obtain the approved runtime relationships for a machine family. Research contributors can inspect candidate status, measurement requirements, calibration diagnostics and falsification plans: observations that would count against a hypothesis. The UI can show readiness and limitations without promoting research candidates into operational authority.

Example and limits: The WS-01 runtime relation compares measured spindle current with a nominal current-speed relationship. It can reveal changed electromechanical load consistency. It does not directly measure cutting force, identify a worn blade or prove a physical root cause. The interpretation must stay attached when residuals, model deviations and fault explanations are built later.

Contributor entry: Failure Research and Reliability: read `steps/step01_physics_library/library.py`, the wafer-saw family and a dossier. First task: explain one failure mode's modeling feasibility, missing measurement or falsification test.

A plausible mechanism is a starting point for an honest experiment. Evidence maturity determines how strongly the project may describe it.

CHAPTER 05 / CURRENT ARCHITECTURE / CONTINUED

STEPS01-03: physics and useful features

STEP02 - summarize a measurement window

STEP02 summarizes a usable measurement window into named features. Each feature describes measured behavior; it is not a health label or failure conclusion. Independent sensor timestamps remain part of the evidence.

What enters

The function receives an exact-machine identity, its station profile, channel windows, operating state and full window start/end times. STEP08 has already stored the asynchronous measurements and assessed their availability. Each channel still carries its own timestamps rather than sharing a fabricated synchronized row.

What happens

`features.py` considers the station's registered channels. A channel needs at least three values and enough coverage for the requested interval; the default minimum is half the expected sample count. An unusable channel is skipped rather than filled with invented measurements.

For each usable channel, the median describes its typical level. Median absolute deviation describes spread around that level. Robust slope compares the median values and median times of the earlier and later halves, describing change per second while reducing the influence of isolated extremes. These are simple, inspectable calculations rather than an unexplained learned representation.

What leaves

A `FeatureSet` contains named features with values, subsystems and kinds: location, spread or trend. It also retains exact-machine identity, operating state and the entire source window. STEP03 adds separate physics features; STEP06 uses the window identity when selecting nominal history; STEP07 compares the features with a baseline.

Example and limits: Consider spindle-current readings 2.0, 2.1 and 8.0. Their median is 2.1, so one extreme reading does not become the typical level. Spread and trend describe other aspects of the window. This arithmetic example is illustrative, not a new expected replay value. In 0.2.6, STEP09 uses eligible location and physics scores for health; displaying a trend does not mean it drives health.

Contributor entry: ML and Data Pipeline: read `steps/step02_physical_features/features.py` and tests. Future first task: explain coverage or protect feature identity and timing.

Never insert the desired health result or a later maintenance event into the current feature values.

CHAPTER 05 / CURRENT ARCHITECTURE / CONTINUED

STEPS01-03: physics and useful features

STEP03 - compare measurements with a physical relationship

STEP03 compares measurements with an approved physical relationship. Their disagreement is a residual. Using it requires supported measurements, permitted calibration and a carefully limited interpretation.

What enters

Inputs are channel windows, the machine family and any previously fitted physics parameters. STEP01 supplies each relation's required channels, units and calculation. Calibration receives permitted nominal measurements; monitoring receives current measurements and the stored parameters from the accepted machine model.

What happens

`residuals.py` finds the interval actually shared by all required signals. It interpolates within supported measurement times only. Interpolation estimates values between supported observations; it must not extend a signal beyond its available time range. Missing channels, wrong units or insufficient common points prevent a residual.

Generic alignment requires 20 overlapping points for fitting and eight at runtime; relations can require more. WS-01 also checks speed variation and calibration range. Monitoring uses the nominal current-speed fit instead of refitting to explain away changed input.

What leaves

The result is a tuple of finite features marked as physics evidence, with subsystem and relation IDs. A relation requiring fitted parameters is skipped if they are unavailable. These features join STEP02 summaries before STEP07 evaluates the exact machine. Calibration parameters are also retained in the model artifact for reproducibility.

Example and limits: If the nominal current-speed relation predicts lower current than is observed under supported conditions, the signed residual describes that disagreement. Later stages can compare its level with nominal behavior and associate elevated evidence with the spindle. The WS-01 residual is an electromechanical consistency measure, not cutting force or causal proof. Different physical or measurement effects can produce disagreement.

Contributor entry: ML, Failure Research and Data Pipeline: read `steps/step03_physical_residuals/residuals.py`, overlap tests and the wafer-saw relation. Future first task: explain unsupported alignment.

Keep calibration and monitoring separate. Missing or out-of-domain physics evidence must not be presented as a supported residual.

CHAPTER 06 / CURRENT ARCHITECTURE

STEPS04-05: optional family-level research

STEP04 - construct a consistent family dataset

STEP04 organizes historical features and event labels from several machines of one family. It supports a different question from the exact-machine path: can earlier measurements help distinguish examples linked to later events?

What enters

A family dataset declares its family, data origin, collection period, samples and observed events. Each sample has a globally unique sample ID, machine ID, timestamp and named feature values. An event has its own ID, machine ID and time. A sample may reference a later event as its training label.

What happens

`dataset.py` checks collection boundaries, finite features, unique sample IDs and consistent feature names across samples. Observed events have unique machine/event keys. A positive sample must reference an existing event on the same machine, and that event must occur after the sample.

Keeping the event reference separate from the input features prevents a basic form of leakage. Leakage means the method receives information that would not have been available when making the prediction. A later maintenance outcome may help label a historical example, but it cannot become a measurement that the running system pretends to know in advance.

What leaves

The dataset exposes ordered feature names and training arrays: measurements, event/non-event labels and machine IDs. Unique-machine and independent-event counts support STEP05 prerequisites. Several windows leading to one event remain one independent event.

Example and limits: Ten earlier windows from machine A referencing event E remain one independent event. Machine B cannot borrow A's event for its label. These checks aid auditing; they do not prove unlabelled windows are healthy or all events represent one failure mechanism.

Contributor entry: Data Pipeline, ML and Reliability: read `steps/step04_family_data/dataset.py` and tests. Future first task: audit an event reference or machine split.

This is an optional historical preparation branch. It does not replace exact-machine nominal history or supply operational authority.

CHAPTER 06 / CURRENT ARCHITECTURE / CONTINUED

STEPS04-05: optional family-level research

STEP05 - fit an advisory family score

STEP05 fits a lightweight model that distinguishes event-linked historical examples from other examples. Its score is advisory research evidence. The deterministic exact-machine path still owns health classification.

What enters

`fit_family_model()` accepts a STEP04 dataset declared `REAL_OSAT`. By default, it requires at least three machines, three independent observed events, six positive windows and both event/non-event classes. For each machine, removing that machine must leave both classes in the remaining training examples.

What happens

`model.py` standardizes the feature values: it stores a training center and scale so differently sized measurements can be compared. It then fits fixed logistic-regression settings with a fixed random seed. The resulting model retains its family, origin, feature names, scaling values and fitted coefficients.

The held-out-machine class-support check tests whether a future split would still have both classes available for training. It does not actually perform a complete held-out evaluation or prove generalization. Reliability must separately test unseen machines or periods, examine false alarms and missed detections, and document event definitions and coverage.

What leaves

`score_family_model()` receives the active family, runtime mode and current named features. It checks family compatibility, permitted origin and complete finite inputs. It returns a machine-wide risk score between zero and one. The pipeline can carry that score beside its exact-machine assessment when the feature schema is available. STEP09 explicitly keeps it from changing deterministic health.

Example and limits: A larger score means the current feature combination looks more like the fitted event-linked examples. It does not mean there is that numerical probability of a future equipment failure. Ranking examples usefully and choosing a reliable operational threshold are separate questions. External benchmark inputs do not gain `REAL_OSAT` family-training status, and the synthetic WS-01 full PoC does not fit this model.

Contributor entry: ML and Reliability: read `steps/step05_family_model/model.py` and tests. Future first task: explain insufficient independent events or design held-out evaluation.

Never rename the advisory score as a calibrated failure probability or use it to override STEP09.

CHAPTER 07 / CURRENT ARCHITECTURE

STEPS06-07: the exact-machine baseline

STEP06 - select permitted nominal history

STEP06 selects historical windows permitted to form one installed machine's nominal reference. It keeps learning from designated nominal data separate from monitoring a possible fault.

What enters

`MachineHistory` receives one exact-machine identity, data origin, historical feature sets and allowed nominal intervals. An interval declares the machine ID, start/end times and permitted operating states. Each feature set already retains its source window and context from STEP02, including any STEP03 residuals.

What happens

`history.py` rejects a history containing feature sets from another exact machine or intervals naming another machine. It requires declared nominal intervals. Selection checks the entire feature window: its start and end must both fall within an allowed interval, and its operating state must be permitted there.

This full-window check matters because a feature is a summary over time. A window ending inside a nominal period can still contain earlier fault measurements. Looking only at its final timestamp would allow excluded behavior into the fitted reference. STEP06 therefore passes only windows whose whole duration is supported by the declaration.

What leaves

`healthy_feature_sets()` returns the accepted historical rows for STEP07 fitting. STEP06 does not discover healthy periods automatically, fit a model or judge a later anomaly. Whoever supplies an interval must justify its designation at the appropriate evidence scope. The retained origin helps later callers distinguish authored synthetic history from real OSAT history.

Example and limits: Suppose an allowed interval begins at 10:00. A one-minute window from 09:59:30 to 10:00:30 overlaps the boundary and is excluded. A fully contained window can qualify if its machine and state also match. This is an illustrative timing example, not a replay checkpoint. The API name `confirmed_healthy` describes the interval field; in the synthetic PoC it means POC-DESIGNATED NOMINAL, not independently confirmed plant health.

Contributor entry: Data Pipeline, ML and Reliability: read `steps/step06_machine_history/history.py` and tests. Future first task: protect a boundary-overlap case and document interval evidence.

A clean timestamp boundary prevents accidental mixing. It does not independently prove the machine was healthy throughout the interval.

CHAPTER 07 / CURRENT ARCHITECTURE / CONTINUED

STEPS06-07: the exact-machine baseline

STEP07 - fit and evaluate the exact machine

STEP07 learns nominal feature levels and variability for one installed machine and state. Evaluation compares current features with that stored reference; monitoring does not silently learn an incoming fault as new normal behavior.

What enters

Fitting receives permitted STEP06 history and calibrated STEP03 parameters. Evaluation receives the fitted model, active exact-machine identity, current feature set and runtime mode. The model belongs to one machine, family and station. PROCESSING and IDLE need their own supported contexts.

What happens

`model.py` groups permitted rows by operating state and keeps features shared across those rows. It needs enough rows per state: the function defaults to 12, while the full PoC requests at least 20 and supplies 35. Its median is the center. Scale uses robust spread, then standard deviation and a small floor to prevent division by zero.

Evaluation calculates the signed difference from the center in units of that scale and maps its magnitude to a bounded anomaly score. Small deviations receive no elevated score. Identity and origin/mode checks remain active; an unavailable context or no matching features returns unavailable model evidence rather than a guessed assessment.

What leaves

Each deviation retains the feature name, current value, nominal center, subsystem, evidence kind, signed standardized difference and score. STEP09 uses eligible scores for health.

`model_io.py` saves a JSON artifact with identity, origin, feature contract, physics parameters, fitted values, validation status and source/calibration/validation identities. Loading checks expectations and reproduces the intended result.

Example and limits: Three stored scales above center means relative change, not three times the failure risk. Unsupported states can produce UNKNOWN. The PoC fits first, purges a full window span and validates separate nominal rows before accepting, saving and reloading the model.

Contributor entry: ML, Data Pipeline and Reliability: read `steps/step07_machine_model/model.py`, `model_io.py` and tests. Future first task: test a wrong-machine load or unsupported context.

Future fault inputs, desired health labels and maintenance outcomes must not influence nominal fitting. Acceptance in this proof remains research-only.

CHAPTER 08 / CURRENT ARCHITECTURE

STEP08: telemetry enters and is stored

STEP08 - accept, store and assess measurements

STEP08 checks which current measurements belong to the machine and whether usable evidence exists. Its input must reach earlier-numbered analysis stages before they can calculate features.

What enters

A source supplies batches of timestamped samples and operating context. `sources.py` owns queued live input and historical replay through `poll()`. `secs_gem.py` maps approved equipment-like values to canonical channels; unknown source IDs and unapproved process-information fields are rejected. Origin and runtime mode stay explicit.

What happens

`store.py` checks exact-machine identity, registered channel, source ID, exact unit, finite value and increasing per-channel timestamps. Context must name the same machine and progress in time. `append_batch()` validates the whole batch before committing any of it. One invalid sample or context cannot leave a partly accepted update behind.

The store keeps separate channel streams because sensors can report at different rates. By default it holds up to 10,000 samples per channel and 2,000 context records in bounded memory. Analysis normally requests a 60-second window. Required channels are checked for missing, stale, future or insufficient measurements; context has a separate 30-second freshness limit.

What leaves

Later stages receive channel windows and a `TelemetryStatus` with validity, observability, usable channels and reasons. Validity concerns usable required measurements. Observability also requires supported current operating context. A required missing channel can block analysis; an unavailable optional channel remains unavailable without an invented value. The store is not a permanent raw-data historian: tickets use STEP11a SQLite and models use STEP07 JSON.

Example and limits: If a batch contains a valid speed reading and current with an unapproved source ID, both are rejected together. In live ingestion, that rejection makes the current result invalid and UNKNOWN rather than retaining cached NORMAL as current evidence. A completed replay can report exhaustion; no new measurements are created to keep its display moving.

Contributor entry: Data Pipeline: read `steps/step08_live_telemetry/sources.py`, `store.py`, `secs_gem.py` and tests. Future first task: protect atomic rejection or independent channel clocks.

Quality and observability are separate from health. Accepting a measurement does not prove a valid model or authorize a maintenance action.

CHAPTER 09 / CURRENT ARCHITECTURE

STEPS09-10: health and fault evidence

STEP09 - deterministic health classification

STEP09 calculates health from supported evidence and prior tracker state. The same permitted inputs and state produce the same result. UI, replay expectations, research reports and language-model wording cannot assign health.

What enters

HealthEngine receives telemetry status, exact-machine results, time, state and mode, plus optional advisory family risk. The station profile identifies subsystems requiring current scoreable evidence.

What happens

health.py groups deviations by subsystem. Location and physics scores are eligible; spread and trend remain visible. The largest eligible score drives each subsystem, then the largest subsystem score drives the machine. Missing required evidence, invalid/unobservable input or an unavailable model produces UNKNOWN.

Persistence requires two seconds before entry and five before improvement. Hysteresis uses different entry/exit levels to limit repeated boundary switching. These frozen rules are not newly validated plant thresholds.

Health state	Entry score	Exit rule from that state
WATCH	At least 0.35	Below 0.25 permits NORMAL
DEGRADED	At least 0.60	Below 0.48 permits WATCH
CRITICAL	At least 0.82	Below 0.70 permits DEGRADED

What leaves

A **HealthAssessment** retains states, deviations, time, context, validity/observability, reasons and transition status. Family risk cannot alter health. STEP10 explains elevated results; the pipeline checks transition eligibility for demo tickets.

Example and limits: A persistent eligible score above 0.60 can enter DEGRADED. Just below 0.60 does not undo it: recovery uses the lower exit level and longer persistence. A large score can skip levels. UNKNOWN means unsupported evidence, not healthy or confirmed faulty.

Contributor entry: ML, Reliability and Data Pipeline: read `steps/step09_health_risk/health.py` and timing tests before a future rule change.

A health state is an evidence classification. It is not a shutdown command, certified diagnosis or failure probability.

CHAPTER 09 / CURRENT ARCHITECTURE / CONTINUED

STEPS09-10: health and fault evidence

STEP10 - structured fault evidence

STEP10 turns elevated health into a traceable explanation of contributing measurements. It describes associated evidence so maintenance wording can refer to actual model outputs instead of inventing a cause.

What enters

`build_fault_evidence()` receives the STEP09 health assessment, including exact-machine identity, runtime mode, timestamp, subsystem states and feature deviations. It only builds fault evidence for DEGRADED or CRITICAL. NORMAL, WATCH and UNKNOWN do not enter this elevated-evidence path.

What happens

`evidence.py` uses the assessment's suspected subsystems to focus on relevant deviations. It keeps deviations with positive scores and orders descriptions with location evidence first, physics evidence next and other kinds afterward. Within those groups, higher scores come first. The current explanation includes up to six named deviations.

Each description retains the feature and subsystem and reports its signed distance from nominal behavior in robust scales. This is the stored baseline scale from STEP07, not a physical unit of damage. The sign says above or below the reference. If no detailed descriptions are available, the fallback says evidence is elevated without causal proof.

What leaves

A `FaultEvidence` record contains identity, time, mode, health state, suspected subsystems and descriptions. It may also carry the explicit limitation of unlocalized advisory family risk. STEP12 uses this structured evidence for retrieval. STEP14 and STEP15 use it for explanation and eligible demo-ticket records. The record does not contain equipment-control instructions or new ticket authority.

Example and limits: In the controlled WS-01 run, actual spindle features and residual deviations support spindle attribution. The PoC does not insert a spindle label as an inference input. Attribution means the spindle-associated evidence contributed to the result; it does not distinguish blade wear, load change, sensor error or another physical mechanism by itself. A qualified reviewer still needs equipment context and approved procedures.

Contributor entry: ML, Reliability, Failure Research and UI: read `steps/step10_fault_evidence/evidence.py` and tests. Future first task: clarify a description without strengthening causality.

Explain what contributed to the result. Do not upgrade an associated subsystem into a confirmed root cause.

CHAPTER 10 / CURRENT ARCHITECTURE

STEPS11-15: maintenance responsibilities

STEP11a - retain maintenance records

STEP11a stores tickets durably in a local database. Records survive the Python process, letting the application find the same machine's active ticket and prior context after a restart.

What enters

The repository receives structured ticket payloads from STEP15 and an explicit SQLite database path under `.artifacts/`. A payload includes ticket ID, machine ID, status, priority, creation/update times and the explanatory evidence. Queries name the exact machine rather than relying on a family-wide match.

What happens

`repository.py` creates the ticket table and a machine/status lookup index when needed. SQLite is an embedded database stored locally, so this proof does not require a separate database server. Save inserts a ticket; update changes an existing ticket and rejects an unknown ticket ID.

Each database operation opens and closes its connection safely. A successful operation commits its changes; an exception rolls them back. Queries use bound values. The repository protects its operations with a lock and limits list/context queries rather than accepting an unbounded result request.

What leaves

`active_for_machine()` returns the most recently updated active ticket for that machine. OPEN, ACKNOWLEDGED and IN_PROGRESS are active statuses. `prior_context()` supplies short historical summaries for retrieval; `list_tickets()` supports the maintenance display. These functions return records, not new health decisions or independently confirmed failure diagnoses.

Example and limits: The controlled run creates a HIGH demo ticket and later escalates the same ticket to URGENT. Reloading the SQLite file returns that same identity and status. Returning to NORMAL does not delete or close it. Historical wording can help explain a current record, but an old ticket is not current sensor evidence and must not be inserted into the nominal model as a known healthy measurement.

Contributor entry: Data Pipeline and UI: read `steps/step11a_maintenance_db/repository.py` and tests. Future first task: protect exact-machine ticket/context isolation.

The repository stores permitted records. STEP15 and the pipeline own the policy deciding whether a demo ticket may be created or updated.

CHAPTER 10 / CURRENT ARCHITECTURE / CONTINUED

STEPS11-15: maintenance responsibilities

STEP11b - load local equipment guidance

STEP11b supplies local equipment guidance for explanations. The bundled text is project-authored research/demo material; the folder's OEM name does not make it OEM-certified guidance or an inference engine.

What enters

`load_oem_manuals()` receives a local JSON file of guidance chunks. A chunk is a small named passage with an ID, equipment-family scope, optional subsystem, title and body. The default file is `resources/maintenance_playbooks.json`. The guidance is supplied locally; this stage does not search the web or download manuals at runtime.

What happens

`manuals.py` converts each chunk into a `ManualChunk`. Loaded body text is explicitly marked RESEARCH/DEMO GUIDANCE. If the selected file does not exist, loading returns no chunks. Other file/format failures are not silently presented as successfully loaded OEM guidance.

`relevant_manuals()` filters by the fault record's machine family, allowing general fleet guidance too. Where a chunk names a subsystem, its scope is checked against the suspected subsystems. These filters reduce irrelevant context before STEP12 ranks passages; they do not establish the physical correctness of a maintenance recommendation.

What leaves

STEP12 receives scoped text records with stable chunk IDs. Their IDs let retrieved passages point back to the local material used for an explanation. The stage does not assign health, close a ticket, change model parameters or send a command to equipment. Local source content still needs a reviewer who understands its evidence and equipment scope.

Example and limits: A spindle-related WS-01 record can retrieve guidance scoped to the wafer-saw family and spindle rather than a wire-bond procedure. General fleet guidance may also qualify. A good scope match does not make a project-authored playbook an approved plant work instruction, and an absent manual is not a reason to erase an otherwise eligible deterministic ticket.

Contributor entry: Failure Research, Data Pipeline and UI: read `steps/step11b_oem_manuals/manuals.py`, its resource and tests. Future first task: clarify a chunk's source and applicability.

Keep source status visible. Retrieved guidance supports an explanation; it does not create operational authority.

CHAPTER 10 / CURRENT ARCHITECTURE / CONTINUED

STEPS11-15: maintenance responsibilities

STEP12 - retrieve passages for an explanation

STEP12 retrieves relevant local text for an existing fault record. RAG means retrieval-augmented generation: find supporting passages before optional writing. Here retrieval is text ranking, not another health model.

What enters

The function receives STEP10 fault evidence, prior maintenance summaries for the exact machine from STEP11a, and local guidance chunks from STEP11b. The pipeline supplies this context only after the deterministic health and fault stages have produced their results.

What happens

`retrieval.py` first filters guidance by the allowed family and subsystem scope. It forms a search query from the machine name, family, suspected subsystems and evidence descriptions. It then uses TF-IDF: a word-weighting method that compares which terms occur in a query and the candidate passages. It considers single words and pairs of words.

The result is ranked by text similarity, with up to four passages by default. Text similarity asks whether wording matches; it does not verify a diagnosis. If there are no candidates, retrieval returns an empty tuple. If the text cannot form a usable vocabulary, it returns a bounded selection rather than inventing content. The pipeline can also fall back if retrieval raises an error.

What leaves

Each `RetrievedPassage` retains a source ID and text. The deterministic fallback can use those passages, and STEP13 can include them in a local wording request. Retrieval returns supporting context; it does not modify the health assessment, feature values, suspected-subsystem decision or permission to issue a demo ticket.

Example and limits: A spindle-current deviation may rank a spindle guidance passage above an unrelated coolant passage. That match explains why the wording was selected, not why the equipment failed. Prior maintenance text may contain useful history but is not new sensor data. No retrieved phrase can grant equipment-control authority or turn an external research result into operational evidence.

Contributor entry: Data Pipeline, Failure Research and UI: read `steps/step12_rag/retrieval.py` and tests. Future first task: check family filtering or preserve source IDs.

Empty or failed retrieval must not suppress an eligible deterministic ticket. The explanation can fall back to the measured evidence.

CHAPTER 10 / CURRENT ARCHITECTURE / CONTINUED

STEPS11-15: maintenance responsibilities

STEP13 - optional local explanation wording

STEP13 optionally asks a local language model to explain existing evidence. The pipeline creates an eligible ticket with deterministic wording first; generated prose can enrich it afterward.

What enters

`generate_local_llm_json()` receives the STEP10 fault record, STEP12 retrieved passages and an optional local model path. The supplied evidence includes machine/family identity, health, suspected subsystems, descriptions and any advisory family-risk limitation. It does not ask the model to infer health directly from raw telemetry.

What happens

`llm.py` checks that a configured file exists and uses the `.gguf` model format. GGUF is a file format used by the optional local inference backend. The stage imports `llama_cpp` only when needed. No configured file, a missing optional package, an unsuitable path or an inference error returns no generated wording.

The prompt asks for exactly three JSON fields: `summary`, `likely_issue` and `recommended_checks`. JSON is structured text with named fields that another function can parse. The prompt restricts the model to supplied evidence and forbids changing health, creating/suppressing tickets or inventing measurements. Those instructions are not scientific verification; STEP14 still checks the returned structure, and deterministic owners retain decisions.

What leaves

The function returns raw text or `None`. Raw text goes to STEP14 rather than directly into a maintenance record. If accepted, it can enrich the existing ticket through STEP15. If absent or invalid, deterministic fallback remains available. Running the ordinary proof does not require a language-model download or successful local inference.

Example and limits: The model might propose a clearer summary of spindle-associated deviations and the supplied local guidance. It cannot change DEGRADED to NORMAL or replace the contributing measurements. The full PoC demonstrates no-LLM fallback and explicitly authored valid/invalid wording fixtures; it does not claim that actual GGUF inference was executed or scientifically validated.

Contributor entry: UI, Data Pipeline and Reliability: read `steps/step13_local_llm/llm.py` and tests. Future first task: check missing-model fallback or clarify evidence restrictions.

Fluent wording can still be wrong. Optional generated prose must never become a source of health, causal or equipment-control authority.

CHAPTER 10 / CURRENT ARCHITECTURE / CONTINUED

STEPS11-15: maintenance responsibilities

STEP14 - validate wording and provide fallback

STEP14 checks the structure of optional generated wording and provides deterministic fallback. Missing or malformed model output must not break the eligible ticket workflow.

What enters

`validate_llm_json()` receives raw STEP13 text or `None`, plus the existing fault evidence and retrieved passages. `deterministic_fallback()` uses the same evidence and context without calling a language model. These functions produce wording records; the fault and health decisions already exist.

What happens

`validation.py` requires a JSON object with exactly `summary`, `likely_issue` and `recommended_checks`. All text fields must be nonempty strings; checks must be a list of one to six strings. Extra fields such as a proposed replacement health state cause fallback rather than being accepted as another control channel.

Accepted strings are cleaned: distracting control and direction-changing characters are removed, whitespace is normalized and lengths are bounded. Summary and likely-issue text are limited to 500 characters; each check to 280. Absent, malformed, wrong-shaped or empty output returns the deterministic fallback.

What leaves

A `TicketEnrichment` contains the summary, likely issue, recommended checks and backend label. The fallback names the elevated health, identifies associated subsystems without causal proof and uses available guidance or a review-with-qualified-personnel instruction. STEP15 records whether wording came from deterministic fallback or the local model.

Example and limits: An output with a valid summary but missing `recommended_checks` is rejected as incomplete. An output with all required fields can pass structural validation. Passing does not prove factual accuracy, a suggested cause or procedure approval. Structural validation is not scientific grounding; retain original evidence and uncertainty.

Contributor entry: Data Pipeline, UI and Reliability: read `steps/step14_json_validation/validation.py` and tests. Future first task: reject extra fields or invisible characters without suppressing tickets.

STEP14 constrains format and supplies fallback. It does not certify the factual accuracy of accepted generated prose.

CHAPTER 10 / CURRENT ARCHITECTURE / CONTINUED

STEPS11-15: maintenance responsibilities

STEP15 - create or update an authorized demo ticket

STEP15 persists eligible deterministic evidence as a demo-only maintenance ticket. A ticket is a review record for a person, not a command to equipment.

What enters

`create_or_update_ticket()` receives the STEP11a repository, STEP10 fault evidence and STEP14 wording. The pipeline calls the ticket path on eligible DEGRADED or CRITICAL transitions. It first supplies deterministic fallback wording, then optionally supplies validated local-model enrichment for that same evidence.

What happens

`tickets.py` permits SIMULATION evidence at this boundary. DEGRADED maps to HIGH priority and CRITICAL to URGENT. If an active ticket already belongs to this machine, the stage updates that record while preserving its ticket ID and creation time. Lower-severity evidence does not downgrade its retained elevated state. Without an active record, it creates an OPEN ticket and saves it through STEP11a.

LIVE_EQUIPMENT and ordinary REAL_REPLAY remain observe-only. The pipeline's narrow exception recognizes the checksum-verified bundled SYNTHETIC reference and bridges it to the same simulation-only demo-ticket stage. That explicit exception does not grant ticket authority to arbitrary replay files or real OSAT input.

What leaves

A `MaintenanceTicket` retains exact machine/station/family identity, status, priority, timestamps, health, associated subsystems, evidence descriptions, wording backend and demo-only flag. The UI reads it and the database persists it. No authorized result returns no newly created ticket; the repository may still retain an older active record.

Example and limits: In the WS-01 proof, DEGRADED creates one HIGH ticket. CRITICAL escalates that same ID to URGENT; optional wording can enrich it without creating another ticket. Recovery to NORMAL leaves the ticket OPEN. The program does not infer that maintenance was performed merely because a signal recovered, and it does not automatically close unresolved work.

Contributor entry: Data Pipeline, UI and Reliability: read `steps/step15_maintenance_ticket/tickets.py`, tests and the pipeline's `_maintenance_ticket()` connection.

Keep inference, wording and authorization separate. Retrieval or language-model failure cannot erase an eligible deterministic ticket or grant a new one authority.

CHAPTER 11 / CURRENT ARCHITECTURE

POST01-02: demonstrations and frozen replay

POST01 - demonstrate a synthetic fleet

POST01 supplies repeatable authored measurements for nine station families and processes them through the operational owners. Synthetic means created for demonstration, not measured from qualified plant equipment.

What enters

`demo.py` uses canonical station profiles and one identified demonstration machine per family. Its telemetry generator follows each channel's independent sampling period and supplies operating context. Nominal generated input is used to prepare demonstration baselines before monitoring the authored fault scenario.

What happens

`SyntheticTelemetrySource` creates batches consumed by the ordinary STEP08 store. The pipeline computes features, residuals, exact-machine deviations, health and fault evidence. The fault control changes generated measurements; it does not tell STEP09 which final label to display.

The nine-machine demonstration supports exploration from the CLI and UI. A selected machine can be monitored or disconnected through demonstration controls. These controls affect the demonstration connection and display; they do not issue commands to industrial equipment. Each machine's telemetry and baseline remain tied to its identity.

What leaves

Readers receive actual pipeline results and permitted demo-only tickets. The dashboard can show telemetry, feature deviations, physics context, health progression and maintenance evidence. The command-line report makes the same controlled behavior accessible without opening PyQt. Generated databases and other runtime artifacts belong under `.artifacts/`.

Example and limits: Selecting WS-01 and applying the authored spindle change lets a student follow changed input into residual and model evidence, elevated health and an associated ticket. Disconnecting should make retained evidence visibly LAST KNOWN. This demonstrates how components work together. It does not prove the injected pattern occurs in a plant, validate every station's failure mechanisms or measure a real false-alarm rate.

Contributor entry: Data Pipeline, UI and ML: read `post_steps/post01_demo/demo.py` and `tests/test_pipeline_demo.py`. Future first task: protect measurement-driven results or explain channel timing.

A reproducible synthetic scenario is valuable functional evidence. Its source and scope must remain visibly synthetic.

CHAPTER 11 / CURRENT ARCHITECTURE / CONTINUED

POST01-02: demonstrations and frozen replay

POST02 - validate a frozen reference, then replay it

POST02 verifies a fixed reference artifact, then processes its telemetry through the operational pipeline. It checks source identity and observed behavior separately, making the result reproducible.

What enters

`artifact.py` loads a local reference directory containing declared identity, station/channel meaning, telemetry, context, checksums and expected observations. The bundled reference is SYNTHETIC evidence played in REAL_REPLAY mode. Its filenames and replay mode do not turn it into plant data.

What happens

Artifact validation checks the pinned bytes and declared contracts before constructing the source. `replay.py` prepares the demonstration model, uses the accepted source and repeatedly calls the ordinary `MachinePipeline.tick()`. Features, residuals, model evaluation, health and fault evidence therefore come from their existing owners.

Expected checkpoints select times at which to inspect results. They are compared with observed output after inference; they do not tell the model or health engine what answer to produce. The report counts valid/invalid ticks, states, residual availability and ticket observations. Replay ends when its source is exhausted rather than generating replacement measurements.

What leaves

The replay returns a reproducible behavior report and the narrow permitted demo-ticket result. Only the checksum-verified bundled synthetic reference receives the pipeline's explicit bridge to simulation ticket policy. Ordinary replay and REAL_OSAT replay remain observe-only. A supplied file cannot gain the exception simply by copying a mode label.

Example and limits: Changing a protected reference byte can fail validation before replay. Changing processing logic can change the observed result and fail a behavior comparison. These are different checks. Snapshot 1 parity tests also compare fleet results, demo/replay payloads, PoC decisions and 27 dashboard views. Narrow branding/report normalization does not rewrite the original expected scientific or decision digests.

Contributor entry: Reliability and Data Pipeline: read `post_steps/post02_reference_replay/artifact.py`, `replay.py` and tests. Future first task: explain a rejected checksum or checkpoint.

A hash checks byte identity; a replay checks behavior. Neither certifies prospective plant performance or a confirmed failure mechanism.

CHAPTER 12 / CURRENT ARCHITECTURE

POST03-04: external scientific evidence

POST03 - descriptive NASA Milling analysis

POST03 describes an external machining dataset offline. It asks what that study supports at its actual scope; its results cannot transfer automatically to semiconductor equipment or enter the operational pipeline.

What enters

`benchmark.py` receives an explicitly supplied local official NASA Milling artifact and the necessary optional parser dependencies. It reads milling signals, experimental conditions and supplied tool-wear measurements. The source is recorded as `EXTERNAL_BENCHMARK`, including its byte identity and source context.

What happens

The loader checks supported local files/archive content and parses the measurement records. The analysis summarizes signals and examines associations with measured flank wear. It also groups conditions and reports confounding: another factor can change both the measured signal and apparent wear relationship.

A correlation says two quantities changed together in the observed data. It does not alone show one caused the other or establish a future detection strategy. Looking within conditions helps the team understand whether an apparent overall pattern could reflect differing experimental settings. Those analyses remain descriptive.

What leaves

The output is a research report with source identity, signal summaries, associations, condition context, coverage and limitations. The headless CLI can write or summarize it. It supplies no OSAT physics calibration, exact-machine baseline, health classification, fault evidence, maintenance ticket or operational HMI result.

Example and limits: A milling signal may increase in records with greater measured wear. That observation may justify a better-controlled research question, but it cannot establish a WS-01 spindle failure threshold. NASA Milling is external non-semiconductor machining data. This release does not download it during execution, and the raw artifact is excluded from release ZIPs. Missing local data is reported as missing rather than a successful experiment.

Contributor entry: Reliability and Data Pipeline: read

`post_steps/post03_external_benchmark/benchmark.py`, its research note and tests. Future first task: explain coverage or a confounder.

A useful external association can guide research. It cannot train an authorized OSAT family model or become an operational health decision by itself.

CHAPTER 12 / CURRENT ARCHITECTURE / CONTINUED

POST03-04: external scientific evidence

POST04 - evaluate each external dataset honestly

POST04 evaluates external datasets with different signals, labels and experimental limits. Its real-data folder name does not grant verified REAL_OSAT status; these research inputs remain EXTERNAL_BENCHMARK.

What enters

A caller selects an explicit dataset ID and local path. Dataset-specific owners declare source identities, signal meaning, label interpretation, mapping, splits and coverage. Raw inputs live under ignored `benchmarks/_external/`, are supplied separately and are not bundled or downloaded during research execution.

What happens

`evaluation.py` dispatches to the relevant owner under `datasets/`. Shared `core/` modules manage source context, reporting and evidence lifecycle; `post_steps/metrics.py` holds pure research calculations. Each dataset keeps its own interpretation rather than forcing incompatible labels into one equipment-health meaning.

Isolated research can reuse STEP07 arithmetic under its own provenance policy. It cannot attach external data to `MachinePipeline`; STEP09, STEP10 and STEP15 are absent. Missing, invalid, inspected and executed inputs have distinct statuses. Unavailable data is not a performance result.

What leaves

Reports retain metrics, splits, coverage, limits and evidence identity. Frozen 0.2.4/0.2.5 bytes/names stay unchanged. Reproduction compares full scientific content and records current evaluator identity separately. Version 0.2.6 does not turn old experiments into new measurements.

Example and limits: ST-AWFD D2 primarily supports external continuous-score discrimination: ranking abnormal examples above normal ones. D1 is fragile descriptive corroboration, with only two abnormal held-out MaterialIDs among 1,807. Neither result transfers or calibrates STEP09 thresholds. TUHH DAD3350 surface evidence remains descriptive surface analysis, not validated operational wafer-saw health prediction. Other entries keep their separate executed, inspected or unavailable scope.

Contributor entry: Reliability and Data Pipeline: read `post_steps/post04_real_data_evaluation/evaluation.py`, one dataset owner, catalog entry and tests. Future first task: explain a split or leakage risk.

External research claims remain distinct from operational qualification.

CHAPTER 13 / CURRENT ARCHITECTURE

POST05: the controlled end-to-end proof

POST05 - coordinate the full functional proof

POST05 checks how existing stages work together under controlled conditions. Fresh synthetic scenarios produce observed decisions and artifact/lineage checks; the proof does not introduce another health algorithm.

What enters

`poc.py` uses the registered WS-01 machine, synthetic scenario inputs, existing PRE/STEP owners and optional local connectivity simulator. Each run starts in a fresh workspace below `.artifacts/` so a previous model or ticket cannot leak into a new proof. The optional simulator has its own pinned dependency profile.

What happens

1. A new exact machine with valid telemetry but no model stays UNKNOWN.
2. Onboarding collects 130 nominal calibration ticks and 35 fitting rows. Their designation is SYNTHETIC / POC-DESIGNATED NOMINAL, not independently confirmed plant health.
3. It purges 61 ticks, evaluates 35 disjoint validation rows, accepts the model as ACCEPTED_RESEARCH_ONLY, then saves and reloads equal STEP07 behavior.
4. Fresh monitoring reaches NORMAL. Changed spindle measurements drive real feature, physics and model evidence through WATCH, DEGRADED and CRITICAL.
5. STEP10 associates the evidence with the spindle. STEP15 creates and escalates one demo ticket. Recovery returns to NORMAL while that ticket remains OPEN.

What leaves

`trace.py` observes completed decisions. `reporting.py` separates functional proof and frozen external science; `lineage.py` checks historical bytes and current identity separately. Limitations and incomplete optional checks stay visible. Success leaves real-OSAT, prospective-plant and production-validation flags false.

Example and limits: Tests change future validation input without changing previously fitted parameters, and compare actual scoring with a counterfactual model scale. This checks that fitting is isolated and the model matters; it does not prove plant accuracy. Expected states and traces never enter inference.

Contributor entry: Data Pipeline owns full-system integration checks; Reliability reviews what their evidence supports. Read `post_steps/post05_full_poc/poc.py`, `scenarios/` and `tests/test_strategy_proof.py`. The next page covers restart, connectivity and verification.

Functional proof does not qualify real OSAT, plant or production use.

CHAPTER 13 / CURRENT ARCHITECTURE / CONTINUED

POST05: the controlled end-to-end proof

POST05 - restart, uncertainty and evidence checks

The full proof also checks restart, UNKNOWN, mapped connectivity and unchanged scientific history. These observations surround the existing operational owners and retain their evidence limits.

Model and ticket reload with deterministic replay

A fresh process reloads the same STEP07 artifact and STEP11a SQLite ticket, then replays the same input. Checks compare model identity, STEP07 output, final health, ticket ID, priority and status. STEP09 rebuilds state from replay. This proves MODEL + TICKET RELOAD WITH DETERMINISTIC REPLAY, not arbitrary live hysteresis continuation.

UNKNOWN is exercised deliberately

Scenarios cover absent model, missing required signal, stale telemetry, invalid intended live batch, unsupported context and connection loss. Unsupported evidence must remain UNKNOWN, without cached NORMAL, invented measurements or a confirmed fault. Explicit uncertainty is part of correctness.

Loopback connectivity has a narrow scope

The pinned optional `secsgem` simulator checks a local equipment-like connection, approved mapped ingestion, invalid/unmapped rejection and disconnect. Only session/report acknowledgments are sent, with no control commands or plant-interoperability claim. Live-style transport does not change synthetic origin.

Evidence and verification remain separate

Current functional results and original external scientific reports have separate identities and interpretation. The preserved historical source ZIP is read-only evidence, not a second imported application. Optional wording fixtures are authored test inputs, not evidence that real GGUF inference ran.

The reviewed source suite ran 323 tests with one optional NASA skip; NASA integration then passed separately. All five snapshot-parity checks, 19 UI tests, physics audit, demo, reference replay, frozen evidence reproduction and full PoC passed. Extracted ZIP tests had five external-data-related skips because raw datasets are excluded.

Contributor entry: Data Pipeline and Reliability: read `scenarios/restart_probe.py`, `scenarios/connectivity.py`, `trace.py`, `reporting.py`, `lineage.py` and tests. Preserve historical pins; distinguish functional correctness from statistical reliability.

Incomplete optional checks must stay visible and cannot pass qualification.

CHAPTER 14 / CURRENT ARCHITECTURE

Execution order: prepare the model

The numbered stage catalog describes owners. This walkthrough describes time: preparation first, repeated monitoring next, then checks of the resulting evidence. It follows the controlled WS-01 onboarding proof in POST05.

- 01 Establish the contract - PRE01-PRE03**
Fix exact-machine identity, signals, units, source origin and policy.
- 02 Choose applicable physics - STEP01**
Use only approved relationships supported by the machine's signals.
- 03 Collect nominal input - STEP08**
Check and store 130 synthetic nominal calibration ticks.
- 04 Calibrate the relationship - STEP03**
Fit physics parameters from usable nominal measurement windows.
- 05 Build permitted history - STEPS02-03, STEP06**
Calculate 35 nominal feature rows with their full window identities.
- 06 Fit this machine - STEP07**
Estimate nominal centers/scales; retain the fitted physics parameters.
- 07 Check separate data - POST05, STEP07**
Purge 61 ticks; evaluate 35 disjoint nominal validation rows.
- 08 Save and reload - STEP07**
Verify identity and equal model results before beginning monitoring.

Why the module numbers recur or jump

Data must exist before it can be analyzed, so STEP08 supplies input to STEPS02-03. STEP03 first fits physics parameters and later uses them. STEP07 first fits a baseline and later compares measurements with it. Reusing these owners keeps fitting and evaluation consistent.

The optional family branch

STEPS04-05 use separately qualified REAL_OSAT family history to prepare an advisory family model. This branch does not replace the exact-machine baseline and is not fitted by the synthetic WS-01 proof.

Preparation can fail. Missing evidence, unsupported context or failed validation must not become an accepted model. The synthetic proof's accepted artifact remains ACCEPTED_RESEARCH_ONLY; it is not plant qualification.

CHAPTER 15 / CURRENT ARCHITECTURE

Execution order: monitor repeatedly

After preparation, repeat this sequence for each monitoring tick. `MachinePipeline.tick()` in `osat_edge/pipeline.py` owns it. The circles below count actions in execution order; STEP labels identify the modules performing those actions.

- 01 Read input - STEP08**
Poll the source and atomically commit an acceptable batch.
- 02 Check evidence - STEP08**
Build current windows; assess quality, observability and context.
- 03 Calculate features - STEPS02-03**
Summarize channels; use stored physics parameters for residuals.
- 04 Compare the machine - STEP07**
Evaluate the fitted model; optionally carry STEP05 advisory risk.
- 05 Determine health - STEP09**
Apply subsystem evidence, persistence and hysteresis.
- 06 Explain elevated evidence - STEP10**
Build structured fault evidence when eligible.
- 07 Maintain the demo ticket - STEPS11-15**
Read context, retrieve passages, use fallback, then create/update.
- 08 Return the result**
UI and POST-STEPS read the completed PipelineResult.

What moves between owners

Action 07 creates the eligible deterministic ticket first. Optional STEP13 wording passes STEP14 validation before STEP15 enriches it. The result carries health, telemetry status, features, fault evidence and any ticket returned this tick; a retained ticket may still exist when none is returned.

What happens when input is unavailable

Insufficient or invalid evidence prevents supported analysis. Missing required signals, stale telemetry, rejected live batches and unavailable model contexts can produce UNKNOWN. A completed replay can report source exhaustion rather than inventing new measurements.

What stays outside this decision

Expected replay checkpoints are observations, not inference inputs. POST05 traces are written after decisions. UI presentation and optional language-model wording cannot assign the health state. External dataset evaluators remain outside the operational pipeline.

CHAPTER 16 / CURRENT ARCHITECTURE

The five screens and headless commands

`osat_edge/ui/dashboard.py` owns the window, selected machine, timers and demonstration controls. Each class under `ui/screens/` owns its own screen widgets and refresh logic. `widgets.py` holds shared cards, trends, tables and formatting. `theme.py` owns shared colors and styles.

Screen	What a contributor or operator can follow
FLEET	The nine station cards, health, monitoring and connection status.
MACHINE	Selected-machine telemetry, source IDs, features and exact-machine deviations.
PHYSICS	Relationships, calibration/evidence context, research readiness and limitations.
MAINTENANCE	Existing tickets, contributing evidence, priority, source context and wording.
SYSTEM	Software version, origin, runtime mode, currentness and authority limits.

Display evidence without strengthening the claim

Text accompanies color. UNKNOWN, unavailable model evidence, research-only physics, uncalibrated risk and demo-only tickets remain distinct. Disconnected displays mark retained assessments as LAST KNOWN rather than presenting them as current measurements.

The intended users are maintenance engineers and factory floor managers. UI reads pipeline and research objects from their owners; it does not fit models, choose health thresholds or grant authority. Plant-specific usability still needs validation. The application uses a single-instance lock.

CLI entry point

`osat_edge/ui/cli.py` exposes `demo`, `reference-replay`, `poc`, `benchmark-nasa-milling`, `evaluate-real`, `verify-real-evidence` and `ui`. Demonstration and research commands can run without starting PyQt's dashboard.

From the extracted project root with the documented environment:

```
.venv\Scripts\python -m osat_edge.ui.cli demo
.venv\Scripts\python -m osat_edge.ui.cli ui
.venv\Scripts\python -m osat_edge.ui.cli poc
```

Offscreen UI tests cover all five screens, current/last-known evidence, research limits and the localized demo-ticket result. The headless PoC produces DecisionTrace; it does not claim to launch the UI itself.

CHAPTER 17 / CURRENT ARCHITECTURE

How the five student teams work together

The five teams share the PRE / STEP / POST architecture. Their responsibilities span stages and depend on clear handoffs.

Semiconductor Failure Research

Research failure modes per OSAT machine, including submachines, physics, sensors and parameters. Explain what should change in measurements, normal lookalikes and uncertainty. Conclude whether each problem could potentially become a computer model, why, and what sensors or evidence are missing. Give Data Pipeline measurement requirements and ML a testable modeling hypothesis. Plausibility alone does not validate a detector.

Data Pipeline & Systems Integration

Own the full pipeline: parsing, integration, consistency and whole-system functional validation. Follow input through analysis, storage and delivery to UI and research. Preserve identity, units, timing, context, provenance, machine separation and whole-batch rejection. Integrate each team's work and test normal and failure paths. Ownership continues after data reaches ML.

Machine Learning & Predictive Maintenance

Build the core computer-run algorithm, including machine learning, for Failure Research's physical problems through Data Pipeline's interfaces. Own features, physical calculations, models and bounded health/fault evidence. Give Reliability the strategy, training sources, split, expected sensitivity and limitations before inspecting held-out results. Preserve deterministic decisions and explicit unavailable cases.

Reliability & Data Research

Investigate full-pipeline outputs for statistical sense and reliability. Check overfitting, leakage, misleading comparisons, confounders, false alarms, missed detections and generalization. Functional success alone does not establish statistical credibility. Return SUPPORTED, REVISE, COLLECT MORE DATA or ABSTAIN with uncertainty and next steps. These research conclusions cannot grant equipment authority.

UI & Data Visualization

Visualize the whole system transparently for maintenance engineers and factory floor managers. Gather OSAT plant requirements; make fleet status, warning evidence, maintenance information and uncertainty easy to follow. Feed user needs back to other teams. Text, charts and color must preserve evidence limits. The controlled PoC has not established plant acceptance.

Human handoff: Failure Research -> Data Pipeline -> ML -> Reliability -> UI. Integration spans all stages; user needs and negative results feed back.

CHAPTER 18 / CURRENT ARCHITECTURE

Find your team's first files

All stage paths below begin at `osat_edge/roadmap/`. Read the owning stage README and one relevant test before editing. Import functions from their concrete owner; there are no old-path compatibility wrappers.

Semiconductor Failure Research

- `STEP01 library.py`, `families/`, `core/schema.py` and `research/` dossiers.
- First task: explain one failure mode's sensors, physics and modeling feasibility.

Data Pipeline & Systems Integration

- `pre_steps/pre01_common/contracts.py`, `PRE02 registry.py`, `PRE03 provenance.py`.
- `steps/step08_live_telemetry/sources.py`, `store.py`, `secs_gem.py`.
- `osat_edge/pipeline.py` connects the full flow; `POST05` checks its integration.
- First task: trace one sample and its identity through input, analysis and display.

Machine Learning & Predictive Maintenance

- `STEP02 features.py`, `STEP03 residuals.py`, `STEP06 history.py`.
- `STEP07 model.py` and `model_io.py`; `STEP05 model.py` for family research.
- `STEP09 health.py` and `STEP10 evidence.py` turn deviations into bounded evidence.
- First task: explain how a calculation addresses a researched physical problem.

Reliability & Data Research

- `POST02 artifact.py` and `replay.py`; `POST03 benchmark.py`.
- `POST04 evaluation.py`, `datasets/` and its research catalog.
- `POST05 poc.py`, scenarios and strategy-proof tests.
- First task: examine a full-pipeline result for leakage, overfitting or confounders.

UI & Data Visualization

- `osat_edge/ui/dashboard.py`, `screens/`, `widgets.py`, `theme.py`, `cli.py`.
- First task: clarify an unavailable/last-known explanation for an engineer or floor manager.

The full concrete rename map is `docs/MODULE_MAP.md`. The five detailed team guides are under `docs/teams/`. Ownership follows responsibility; coordinate with the adjacent team when a shared contract changes.

CHAPTER 19 / CURRENT ARCHITECTURE

Repository boundaries, checks and release identity

Where files belong

`osat_edge/roadmap/pre_steps/` holds trust contracts. `steps/` holds scientific and maintenance owners. `post_steps/` holds demonstrations and research checks. `osat_edge/ui/` holds presentation. `docs/` holds contributor guidance, editable PDF content, style and the changelog.

`.artifacts/` holds generated models, SQLite files, nominal histories, simulator output and verification logs. `.venv/` is the local Python environment. External raw datasets belong under ignored `benchmarks/_external/`. These local/runtime files do not belong in a portable release.

The documented environment

Python 3.12 is supported. Root `requirements.txt` pins NumPy, scikit-learn and PyQt6. Research, surface-parser, simulator and local-LLM dependency profiles remain owned by the relevant stages. Do not add optional research tooling to the runtime merely to run a different task.

What to check before a code handoff

Run targeted boundary tests, the required ResourceWarning-strict full suite and `compileall`. Scientific changes require relevant physics/research checks and honest evidence reproduction. Verify demo, reference replay, full PoC, UI where available and the extracted release ZIP. A documentation-only addition still requires document and archive checks.

Historical source and current source are different records

POST05's `frozen_025_sources.zip` preserves 112 original pinned files. It is read-only evidence, never an imported second implementation. Current implementation identity is separately pinned. Historical reports and hashes are not rewritten to match a newer organization or product name.

Current frozen 0.2.6 implementation identity:

```
20c3594c9cf5a31720a95300721fb7721a6dd51f583e4ba6f317cd4e0adf370b
```

Documentation accompanies every future version

Update only affected guide sections and append the new version's changes to the changelog. Keep previous entries and section IDs. Regenerate and visually check the PDF, then include it directly inside the ZIP's project root. The exact maintenance procedure is `docs/PDF_RELEASE_WORKFLOW.md`.

CHAPTER 20 / CURRENT ARCHITECTURE

A short glossary

Term	Plain-English meaning
Telemetry	Timestamped measurements reported by a machine or a clearly labeled simulator.
Canonical channel	The approved project name and meaning for a measurement.
Exact machine	One identified installed unit, with its own machine ID, family and station.
Family	Equipment grouped by a common role; it is not an exact-machine identity.
Operating context	What the machine is doing, such as PROCESSING or IDLE.
Feature	A named summary or calculated quantity derived from usable measurements.
Nominal baseline	The permitted reference behavior used for comparison.
Residual	Difference between a measurement and what a relationship predicts.
Calibration	Estimating relationship parameters from permitted reference measurements.
Holdout / validation	Data set aside from fitting to check a method separately.
Leakage	Information reaching fitting or testing that should have been unavailable.
Observability	Enough usable current evidence exists to support the intended analysis.
UNKNOWN	Current evidence does not support a health classification.
Persistence	A condition must continue long enough before a transition.
Hysteresis	Different entry and exit levels reduce unstable boundary switching.
Provenance / lineage	Recorded source identity and the path by which evidence was produced.
SHADOW / READ-ONLY	Observe and report within policy, without equipment-control authority.
RAG	Retrieve relevant local documents to support explanations.
Deterministic	The same permitted inputs and state produce the same result.
Demo ticket	A maintenance record permitted only by the demonstration policy.

For deeper wording and related concepts, read `docs/GLOSSARY.md` and the owning stage README. If a term is unclear, improve the explanation at its owner rather than inventing another name for the same concept.

CHAPTER 21 / VERSION HISTORY

Changelog from 0.2.6

This cumulative history begins at 0.2.6. Earlier immutable evidence remains in its original records; those releases are not reconstructed or rewritten here.

0.2.6 - frozen code baseline - 2026-09-30

Structure and contributor navigation

- Accepted snapshot 2 keeps PRE-STEPS / STEPS / POST-STEPS, descriptive filenames and direct ownership without compatibility wrappers.
- STEP08 separates sources, storage and mapping. STEP07 keeps model and artifact owners together. Five UI screens, shared widgets/theme, stage READMEs and team guides clarify contributor entry points.

Controlled proof and current branding

- The current name is OSAT SemiGuard; `osat_edge` and historical evidence identities are retained.
- The controlled proof covers nominal onboarding, actual model/physics monitoring, health progression, spindle evidence, one escalating OPEN demo ticket, UNKNOWN and optional loopback connectivity.
- Restart qualification now checks model identity, final health, STEP07 output and ticket identity/priority/status. This strengthened proof checks without changing scientific inference.
- Reporting separates functional proof from external evidence and states MODEL + TICKET RELOAD WITH DETERMINISTIC REPLAY accurately.

Evidence and verification

- Equations, model math, health thresholds/hysteresis, station definitions, fault/ticket authority, reference replay and historical evidence are preserved.
- Verification passed: 323 source tests with one optional NASA skip subsequently exercised; physics, UI, parity, demo, replay and frozen evidence checks. Real OSAT, prospective plant and production-validation flags remain false.

Documentation edition 1 - no application-code change

- Added the root-level architecture PDF, editable source, stable section IDs, documentation renderer and cumulative history. Existing application code, tests, resources, dependencies and snapshot archives are unchanged.
- Standing policy: every code change updates affected PDF content and the version history; every release includes the visually verified PDF at its project root.

Documentation edition 2 - reading order - 2026-09-30

- Reordered the stage catalog to PRE01-PRE03, STEP01-STEP15 and POST01-POST05. Added separate preparation and monitoring walkthroughs; aligned both team pages with the evidence handoff. Affected sections: S01-S02, S09, S11, S17; new S21. Other stage descriptions retain their text and identities.
- Reader chapters now count upward independently of stable internal section IDs. Preserved the approved visual style and all frozen application bytes; updated the guide and release ZIP only.

Documentation edition 3 - expanded stage explanations - 2026-09-30

- Expanded all 24 stage descriptions: PRE01-PRE03, STEP01-STEP15 including separate STEP11a/11b, and POST01-POST05. Each explains purpose, inputs, processing, outputs, examples, evidence limits and contributor entry. Affected internal sections: S03, S05-S08, S10-S11, S13-S15; S01 reader guidance updated.
- Added stage pages and nested stage bookmarks within the established chapters. Kept numerical catalog order, execution walkthroughs, stable section IDs, fonts, colors, margins and approved visual hierarchy. Preserved unaffected architecture text and all prior changelog entries.
- Clarified current boundaries, including research-only model acceptance, UNKNOWN, replay/demo ticket authority and structural wording validation without factual certification. This is a documentation-only edition: frozen application code, tests, resources, dependencies and historical snapshots are unchanged.

Documentation edition 4 - full team responsibilities - 2026-09-30

- Clarified Failure Research's per-machine failure inventory, physics, submachines, sensors, parameters and modeling-feasibility conclusion. Defined ML ownership of the core computational algorithm, including machine learning, built through the shared pipeline.
- Expanded Data Pipeline responsibility to full integration, parsing, consistency and whole-system functional validation. Expanded Reliability to statistical investigation of complete pipeline outputs, including overfitting, leakage, confounders and unsupported interpretations. Functional correctness and statistical credibility remain separate, complementary questions.
- Defined transparent whole-system UI for maintenance engineers and factory floor managers, guided by OSAT plant requirements without claiming completed plant acceptance. Updated only affected guide sections S06, S12, S15-S17, the six team guides and the student handoff summary. Preserved chapter/stage order, visual formatting, prior history, frozen application bytes and historical snapshots.